

Question No:5

Question description

You have an undirected graph consisting of n vertices with weighted edges.

A simple cycle is a cycle of the graph without repeated vertices. Let the *weight* of the cycle be the **XOR** of weights of edges it consists of.

Let's say the graph is *good* if all its *simple* cycles have weight 1. A graph is bad if it's not good.

Initially, the graph is empty. Then q queries follow. Each query has the next type:

- $u\ v\ x$ — add edge between vertices u and v of weight x if it doesn't make the graph bad.

For each query print, was the edge added or not.

Constraints

$3 \leq n \leq 3 \cdot 10^5$; $1 \leq q \leq 5 \cdot 10^5$

$1 \leq u, v \leq n$; $u \neq v$; $0 \leq x \leq 1$

Input

The first line contains two integers n and q — the number of vertices and queries.

Next q lines contain queries — one per line. Each query contains three integers u , v and x — the vertices of the edge and its weight.

It's guaranteed that there are no multiple edges in the input.

Output

For each query, print YES if the edge was added to the graph, or NO otherwise (both case-insensitive).

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN)	INPUT (STDIN)
9 12	9 6
6 1 0	1 3 1
1 3 1	3 6 0
3 6 0	6 4 1
6 2 0	2 4 0
6 4 1	4 5 0
3 4 1	7 8 1
2 4 0	
2 5 0	EXPECTED OUTPUT
4 5 0	YES
7 8 1	YES
8 9 1	YES
9 7 0	YES
EXPECTED OUTPUT	
YES	
YES	
YES	
YES	
NO	
YES	
YES	
NO	
YES	
YES	
NO	

Code:

```
1  #include <stdio.h>
2
3  #define MAXN 300005
4
5  int parent[MAXN];
6  int xr[MAXN]; // XOR from node to its parent
7  int hasCycle[MAXN]; // 1 if component already has a cycle
8
9  /*
10 |   Find root and calculate XOR from x to root.
11 */
12 int find(int x)
13 {
14     if (parent[x] == x)
15         return x;
16
17     int p = parent[x];
18
19     parent[x] = find(p);
20     xr[x] ^= xr[p];
21
22     return parent[x];
23 }
24
25 /*
26 |   Add edge (u, v) with weight x.
27 */
28 int addEdge(int u, int v, int x)
```

```

29 {
30     int ru = find(u);
31     int xu = xr[u];    // XOR u -> root
32
33     int rv = find(v);
34     int xv = xr[v];    // XOR v -> root
35
36     /*
37      Different components:
38      Adding the edge cannot create a cycle.
39     */
40     if (ru != rv)
41     {
42         /*
43          We need:
44
45          XOR(u -> rootU)
46          XOR x
47          XOR XOR(v -> rootV)
48
49          to represent the relation between roots.
50         */
51         parent[rv] = ru;
52         xr[rv] = xu ^ x ^ xv;
53
54         /*
55          If either component already has a cycle,

```

```

56         the merged component has a cycle.
57     */
58     hasCycle[ru] = hasCycle[ru] || hasCycle[rv];
59
60     return 1;
61 }
62
63 /*
64     Same component.
65
66     If it already has a cycle, adding another
67     edge creates another independent cycle,
68     so the graph becomes bad.
69 */
70 if (hasCycle[ru])
71     return 0;
72
73 /*
74     XOR of the existing path u -> v:
75     xu XOR xv
76
77     New cycle XOR:
78     xu XOR xv XOR x
79     xu XOR xv XOR x
80
81     It must be exactly 1.
82 */
83 if ((xu ^ xv ^ x) != 1)
84     return 0;
85
86 /*
87     This edge creates the first cycle in
88     this component, and that cycle has XOR 1.
89 */
90 hasCycle[ru] = 1;
91
92 return 1;
93 }
94
95 int main()
96 {
97     int n, q;
98
99     scanf("%d %d", &n, &q);
100
101     /*
102         Initially every vertex is its own component.
103     */

```

```
103     for (int i = 1; i <= n; i++)
104     {
105         parent[i] = i;
106         xr[i] = 0;
107         hasCycle[i] = 0;
108     }
109
110     for (int i = 0; i < q; i++)
111     {
112         int u, v, x;
113
114         scanf("%d %d %d", &u, &v, &x);
115
116         if (addEdge(u, v, x))
117             printf("YES\n");
118         else
119             printf("NO\n");
120     }
121
122     return 0;
123 }
```

Output:

Test Case-1:

```
CE\api3.exe
9 12
6 1 0
YES
1 3 1
YES
3 6 0
YES
6 2 0
YES
6 4 1
YES
3 4 1
NO
2 4 0
NO
2 5 0
YES
4 5 0
NO
7 8 1
YES
8 9 1
YES
9 7 0
NO
PS C:\Users\kesha\OneDrive\Documents>
```

Test Case-2:

```
CE\api3.exe
9 6
1 3 1
YES
3 6 0
YES
6 4 1
YES
2 4 0
YES
4 5 0
YES
7 8 1
YES
PS C:\Users\kesha\OneDrive\Documents>
```

Keshav Kumar Rauniyar

CH.SC.U4CSE25236